

MSM Double-Counting Bug Fix Summary

Bug Description

Issue: Memory leak statistics were being inflated due to double-counting in the `msm_detect_leaks` function.

Location: `tools/crrss/msm/msm.c` - `msm_detect_leaks` function (lines 1253-1291)

Problem: Every time `msm_detect_leaks` was called (e.g., when generating a report via `msm_generate_report`), it would increment `ctx->stats.memory_leaks_detected` for every still-allocated block. This meant calling `msm_generate_report` multiple times would inflate the leak counter even when no new allocations occurred, degrading the safety score incorrectly.

Root Cause

```
// In msm_detect_leaks function (BEFORE FIX)
if (meta && !meta->is_freed) {
    leaks[*num_leaks] = *meta;
    (*num_leaks)++;

    pthread_mutex_lock(&ctx->stats_lock);
    ctx->stats.memory_leaks_detected++; // BUG: Increments every time!
    pthread_mutex_unlock(&ctx->stats_lock);
}
```

Each call to `msm_detect_leaks` would count all unfreed allocations again, leading to:

- First call: 3 leaks detected → counter = 3
- Second call: 3 leaks detected → counter = 6 (should be 3!)
- Third call: 3 leaks detected → counter = 9 (should be 3!)

Fix Implementation

Approach

Added a tracking flag to the `allocation_metadata_t` structure to mark allocations that have already been counted as leaks. The fix ensures each allocation is counted exactly once, regardless of how many times leak detection runs.

Changes Made

1. Updated `allocation_metadata_t` structure (`msm.h`)

File: `tools/crrss/msm/msm.h` (lines 166-173)

```
typedef struct {
    // ... existing fields ...

    // Access tracking
    uint64_t read_count;
    uint64_t write_count;
    struct timespec last_access_time;

    // Leak tracking
    bool counted_as_leak; // Flag to prevent double-counting in statistics
} allocation_metadata_t;
```

2. Initialize the flag (msm.c)

File: tools/crrss/msm/msm.c (line 293)

```
static allocation_metadata_t* create_allocation_metadata(...) {
    allocation_metadata_t* meta = calloc(1, sizeof(allocation_metadata_t));
    if (!meta) return NULL;

    meta->address = address;
    meta->size = size;
    meta->is_freed = false;
    meta->counted_as_leak = false; // Initialize leak tracking flag

    // ... rest of initialization ...
}
```

3. Modified leak detection logic (msm.c)

File: tools/crrss/msm/msm.c (lines 1275-1289)

```
// If not freed, it's a potential leak
if (meta && !meta->is_freed) {
    leaks[*num_leaks] = *meta; // Copy metadata
    (*num_leaks)++;

    // Only increment the counter if this leak hasn't been counted yet
    if (!meta->counted_as_leak) {
        pthread_mutex_lock(&ctx->stats_lock);
        ctx->stats.memory_leaks_detected++;
        pthread_mutex_unlock(&ctx->stats_lock);

        // Mark as counted to prevent double-counting in future calls
        meta->counted_as_leak = true;
    }
}
```

How It Works

- First Detection:** When a leak is detected for the first time, `counted_as_leak` is false, so we:
 - Increment the statistics counter
 - Set `counted_as_leak = true`
- Subsequent Detections:** When the same leak is detected again, `counted_as_leak` is true, so we:
 - Still include it in the leak array (it's still a leak!)
 - Skip incrementing the counter (already counted)

3. **Freed Allocations:** If an allocation is freed, it won't be detected as a leak anymore (the check `! meta->is_freed` ensures this)
 4. **New Allocations:** Each new allocation gets a fresh metadata object with `counted_as_leak = false`
-

Test Coverage

New Tests Added

Added two comprehensive tests to verify the fix works correctly:

Test 1: `test_leak_double_counting_prevention`

Purpose: Directly tests `msm_detect_leaks` function

Test Scenario:

- Track 3 allocations without freeing
- Call `msm_detect_leaks` three times
- Verify leak count remains 3 across all calls

- ✓ First detection: 3 leaks, counter = 3
- ✓ Second detection: 3 leaks, counter = 3 (not 6!)
- ✓ Third detection: 3 leaks, counter = 3 (not 9!)

Test 2: `test_leak_counting_with_report_generation`

Purpose: Tests the bug scenario via report generation

Test Scenario:

- Track 2 allocations without freeing
- Generate report 3 times (each calls `msm_detect_leaks`)
- Verify leak count remains 2 across all reports

- ✓ First report: 2 leaks detected, counter = 2
- ✓ Second report: 2 leaks detected, counter = 2 (not 4!)
- ✓ Third report: 2 leaks detected, counter = 2 (not 6!)

Test Results

All tests passing: 27/27 (100%)

```

=====
MSM Test Suite
=====

[TEST] MSM Initialization ... ☒ PASS
[TEST] MSM Invalid Initialization ... ☒ PASS
[TEST] MSM Reset ... ☒ PASS
[TEST] Allocation Tracking ... ☒ PASS
[TEST] Deallocation Tracking ... ☒ PASS
[TEST] Double-Free Detection ... ☒ PASS
[TEST] Pointer Tracking ... ☒ PASS
[TEST] Pointer Validation ... ☒ PASS
[TEST] Use-After-Free Detection ... ☒ PASS
[TEST] Use-After-Free Static Detection ... ☒ PASS
[TEST] Double-Free Static Detection ... ☒ PASS
[TEST] NULL-Check Analysis ... ☒ PASS
[TEST] Buffer Overflow Detection ... ☒ PASS
[TEST] Buffer Access Checking ... ☒ PASS
[TEST] Memory Leak Detection ... ☒ PASS
[TEST] Memory Leak Static Analysis ... ☒ PASS
[TEST] Leak Double-Counting Prevention ... ☒ PASS
[TEST] Leak Counting with Multiple Report Generations ... ☒ PASS
[TEST] Comprehensive File Analysis ... ☒ PASS
[TEST] Code Snippet Analysis ... ☒ PASS
[TEST] Statistics Generation ... ☒ PASS
[TEST] Report Generation ... ☒ PASS
[TEST] Safety Score Calculation ... ☒ PASS
[TEST] Issue Query by Type ... ☒ PASS
[TEST] Issue Query by Priority ... ☒ PASS
[TEST] Integration Setup ... ☒ PASS
[TEST] Utility Functions ... ☒ PASS

=====
Test Summary
=====

Total Tests: 27
Passed: 27 (100.0%)
Failed: 0
=====

```

Impact Analysis

Benefits

1. **Accurate Statistics**: Leak counts now reflect unique leaks, not detection frequency
2. **Correct Safety Scores**: Safety score calculations are based on accurate statistics
3. **No Breaking Changes**: Fix is backward compatible - all existing tests pass
4. **Thread-Safe**: Uses existing mutex locks for thread safety
5. **Minimal Overhead**: Only adds one boolean flag per allocation (negligible memory impact)

Before vs After

Before Fix (Bug Present):

```
Track 2 allocations
Generate Report #1 → leak_count = 2
Generate Report #2 → leak_count = 4 ✗ (WRONG!)
Generate Report #3 → leak_count = 6 ✗ (WRONG!)
Safety Score = 0.3 (incorrectly degraded)
```

After Fix:

```
Track 2 allocations
Generate Report #1 → leak_count = 2
Generate Report #2 → leak_count = 2 ✓ (CORRECT!)
Generate Report #3 → leak_count = 2 ✓ (CORRECT!)
Safety Score = 0.8 (accurate)
```

Commit Details

Branch: feature/crrss-tooling-stage2**Commit:** 436b9cf**Message:**

```
fix(crrss/msm): Prevent double-counting of memory leaks in statistics

- Added 'counted_as_leak' flag to allocation_metadata_t structure
- Initialize flag to false when creating allocation metadata
- Modified msm_detect_leaks to only increment leak counter on first detection
- Prevents statistics inflation when msm_detect_leaks or msm_generate_report
  is called multiple times
- Added comprehensive tests to verify the fix:
  * test_leak_double_counting_prevention: Tests direct msm_detect_leaks calls
  * test_leak_counting_with_report_generation: Tests via report generation
- All existing tests continue to pass (27/27 tests passing)

This fix ensures accurate leak statistics and safety score calculations by
ensuring each unfreed allocation is counted exactly once as a leak, regardless
of how many times detection or reporting functions are called.
```

Files Changed:

- tools/crrss/msm/msm.h (+3 lines)
- tools/crrss/msm/msm.c (+7 lines)
- tools/crrss/tests/test_msm.c (+132 lines)

Total: 3 files changed, 142 insertions(+), 3 deletions(-)

PR Update

The fix has been pushed to the feature/crrss-tooling-stage2 branch and will be included in **PR #169**.

Verification Steps for Reviewers

1. Compile and run tests:

```
bash
cd tools/crrss
gcc -o test_msm tests/test_msm.c msm/msm.c common/crrss_types.c -I. -Icommon -pthread -lm
./test_msm
```

Expected: All 27 tests should pass

2. Verify the fix manually:

```
```c
msm_context_t* ctx = msm_initialize(&config);

// Track allocations
msm_track_allocation(ctx, addr1, 100, "test.c", 10, "test");
msm_track_allocation(ctx, addr2, 200, "test.c", 20, "test");

// Generate multiple reports
msm_generate_report(ctx, &report1);
msm_generate_report(ctx, &report2);
msm_generate_report(ctx, &report3);

// Verify leak counts are identical
assert(report1.statistics.memory_leaks_detected == 2);
assert(report2.statistics.memory_leaks_detected == 2);
assert(report3.statistics.memory_leaks_detected == 2);
```
```

1. Check safety score stability:

- Safety scores should remain stable across multiple report generations
- No artificial degradation due to leak count inflation

Conclusion

This fix successfully resolves the double-counting bug in the MSM implementation, ensuring accurate memory leak statistics and reliable safety score calculations. The solution is:

-  **Correct:** Prevents double-counting while preserving all leak information
-  **Efficient:** Minimal memory overhead (1 bool per allocation)
-  **Tested:** Comprehensive test coverage with 100% pass rate
-  **Safe:** Thread-safe implementation using existing locks
-  **Compatible:** No breaking changes to existing functionality

The fix is ready for review and merge into the main branch.